

カンペ（発表者用）－ 第2回研究会発表

発表者だけが見る台本。各スライドの「話す内容」と「★強調」をまとめたもの。

全体の目安：約15～16分（質疑別）。聴衆＝教授＋他分野の学生（前提知識は少なめと想定）。

※ 今回の構成は「①論文解説 → ②経過報告」の2部。冒頭で提案手法の土台となる論文（永原 2017）を理論＋自作コードで解説し（スライド3～10、約8～9分）、続いて実機での実装と評価を経過報告する（スライド11～24）。補足のコード解説（25～32）は時間に応じて取捨選択。

※ 論文解説は学術発表として淡々と進める。経過報告では日程の話はしない（聞かれたら口頭で「次にやる」程度）。

開始時の一言

- ・「制御の“手数”を最小限にする研究です」と最初に一言で言い切る。
- ・専門用語はこの後ていねいに説明する、と予告して安心させる。

スライド1 表紙〔目安 0:20〕

話す：

- ・第2回研究会発表を始める。テーマは農業用ロボットの経路追従におけるスパース最適制御。
- ・前回の発表以降にやってきたことを、論文解説とあわせて報告する。

★強調（自分用）：

- ・★ 最初の一言で『制御の手数（操舵）を最小限にする研究です』と言い切る。
- ・★ 聴衆は他分野の学生もいる前提で、専門用語はこの後ていねいに説明すると予告する。

スライド2 目次〔目安 0:20〕

話す：

- ・構成は2部。まず研究の土台になる論文を解説し（理論＋コード）、つぎに実機での実装と評価を経過報告する。
- ・最後にまとめ。補足としてプログラムの詳細も用意した（時間に応じて）。

★強調（自分用）：

- ・★ 『論文解説（理論）と経過報告（実機）の2部』と最初に枠組みを示す。
- ・★ 論文解説は今回しっかり時間を取るパート、と予告する。

スライド3 論文解説（章扉）〔目安 0:25〕

話す：

- ・まず、研究の土台になっている論文を解説する。永原先生の「最適制御とスパースモデリング」。
- ・理論だけでなく、自分が書いたコードと結びつけて、はじめての人にも分かるように話す。

★強調（自分用）：

- ・★ 「ここが今回の“論文解説”パート」と区切りを明示。身構えさせない。
- ・★ 自分もまだ勉強中で、実装しながら理解している最中、と正直に前置きしてよい。

スライド4 論文① 最大ハンズオフ制御の概要と位置づけ〔目安 1:00〕

話す：

- ・著者の永原先生は「最大ハンズオフ制御」を提案した第一人者。本研究のテーマそのもの。
- ・一言でいうと「できるだけ手を放す（モータを止める）」ように制御を設計する理論。
- ・発想の出どころは制御の“外”。信号処理の「圧縮センシング／スパースモデリング」から来た。
- ・動機は省エネルギー。止める＝電気も燃料も使わない。

- 右の流れ：信号処理→制御→本研究、と橋渡しされてきた。自分の予備実装（操舵78%ゼロ）はこの離散時間版に相当する。

★強調（自分用）：

- ★ 「制御の“外” から来たアイデア」は他分野の人に刺さる。
- ★ 自分の研究が論文の系譜に乗っていることを示す。

スライド5 論文② 従来の最適制御との差異 【目安 1:10】

話す：

- 従来の最適制御（L2）は、エネルギー（入力の2乗）を最小化＝なめらかで常に少し動く。手を添えっぱなし。
- スパース（L1/L0）は、動かす「手数」を最小化＝多くの時間はゼロ、要所だけ最大で動かす。
- 評価軸そのものが逆：「いかに“うまく”動かすか」ではなく「いかに“動かさずに”済ませるか」。
- 右図：どちらも目標には到達する。違うのは“入力の使い方”。

★強調（自分用）：

- ★ ここが差別化の核心。手ぶりで「ずっと添える」vs「パッと放す」を表現すると伝わる。
- ★ 右図は『模式図』と断る（実データは後のスライドで出す）。

スライド6 論文③ 入力の疎性と L0 ノルム 【目安 1:10】

話す：

- 測りたいのは「どれだけ動かしたか」ではなく「動かした時間（回数）」。それがL0ノルム＝ゼロでない要素の個数。
- L0を最小化＝OFFの時間を最大化＝最大ハンズオフ制御。
- 右図：罰し方の“形”が結果を決める。L2は0付近がなめらかで0にしない／L1は0で折れて0が出る／L0は理想。
- 省エネルギーの理由：止まっている間はエネルギーを使わない。

★強調（自分用）：

- ★ 「動かす回数（手数）」という見方で直感をつくる。
- ★ 「L0が理想だが解けない」で次のスライドへ引っぱる。

スライド7 論文④ L0 最小化の緩和 — L1 ノルムの利用 【目安 1:10】

話す：

- L0最小化は「動かす瞬間の組合せ」を総当たり＝組合せ爆発でほぼ解けない（NP困難）。
- そこで圧縮センシングと同じ一手：L0をL1に置き換える。L1なら凸最適化として速く・確実に解ける。
- なぜL1だと0が増える？右図：制約に一番近い点が、L1だと“角”＝軸の上で当たるから、片方がちょうど0になる。
- 数学的には「不感帯（小さい入力では0に潰す）」として現れる。論文の図5に当たる。

★強調（自分用）：

- ★ 「理想は解けない→凸問題に緩和して代用する」という筋で運ぶ。
- ★ 図の“角でぶつかる”を指でなぞる。

スライド8 論文⑤ 主要結果 — L1 最適性と L0 最適性の一致 【目安 1:10】

話す：

- 論文の一番大事な結果：ある自然な条件（正規性）のもとで、L1最適制御＝L0最適（最大ハンズオフ）制御。
- つまり、解ける凸問題（L1）を解くだけで、本当に一番疎な制御が手に入る。

- 解の形は bang-off-bang : +最大/0/−最大 の3値だけ。0の区間が、まさに手放し。
- 「滑らかに少し動かす」のではなく「止める or 振り切る」が最適、という制御。

★強調（自分用）：

- ★ ここが理論のハイライト。「解けるL1が、理想のL0と一致する」と言い切る。
- ★ 数式は出さず、結論と右図（3値）で伝える。

スライド9 論文⑥ 離散時間定式化と実装 [目安 1:20]

話す：

- 論文の数式は、終端で誤差 $0 = \Phi z = \zeta$ という1本の線形制約に書ける。解くのは $\min \|z\|_\infty \text{ s.t. } \Phi z = \zeta, \|z\|_\infty \leq 1$ 。
- これを cvxpy でほぼ数式のまま書いた（左のコード、★がL1の1行）。
- 右が結果。操舵は $\pm$ 最大 か 0 だけ = bang-off-bang、78%が厳密に0 = hands-off。論文の理論をローバーの運動モデルで再現できた。

★強調（自分用）：

- ★ 「理論→コード→実機のモデル」が一枚で繋がるのが見せ場。
- ★ 右図の“ $\pm$ 最大と0だけ”を指さす（論文のbang-off-bangの実物）。

スライド10 論文⑦ MPC への拡張と本研究の位置づけ [目安 1:10]

話す：

- 論文の制御はオープンループ（最初に全部計算）なので、外乱で誤差が溜まると弱い。
- 本研究はそれを、毎周期解き直すMPCに発展させ、補正 δu のL1を最小化した（右コード、★の1行）。
- 理論と実装を往復して分かったこと：simでは論文どおり78%ゼロを再現できたが、実機ではL1が遅延でチャタった。理想と現実のギャップは“実装して初めて”見えた。
- 「実装しながら理論を理解する」、この往復こそが研究の中身。

★強調（自分用）：

- ★ 論文の先＝自分の貢献（MPC化）に繋ぐ。
- ★ 締めは「実装して初めて分かった」。経過報告のまとめ（スライド24）とも響き合う。

スライド11 経過報告（章扉） [目安 0:20]

話す：

- ここからは経過報告。研究の背景・目的から、実機での実装と3手法（Kanayama/L2/L1）の比較・評価までを順に話す。

★強調（自分用）：

- ★ 「理論（論文解説）を、ここから実機でどう確かめたか」という橋渡しを一言。

スライド12 研究の背景と目的 [目安 1:10]

話す：

- まず『なぜこの研究か』。農業は人手不足で、自動走行ロボットが求められている。
- その基礎が経路追従＝決めた道をなぞる制御。
- 普通の制御は常時こまめに操舵するが、モータが消耗し、電気も食い、滑りやすい畑では小刻みな操舵が逆に乱れを生む。
- だから『必要な時だけ操舵する＝スパース制御』に注目した、という流れで話す。

★強調（自分用）：

- ★ “スパース＝時間的にスカスカ”を、身ぶりで（ハンドルをほとんど切らないイメージで）伝えと他分野でも伝わる。

- ・★ ここで聴衆の『なぜ?』を解消しておく、と、実機の話がスッと入る。

スライド13 研究の目的と進め方 [目安 1:10]

話す：

- ・目的は、操舵を時間的に疎にする手法を“実機で”評価すること。
- ・手法の中心はMPCとL1。MPCは『少し先を予測して最適な入力を毎周期計算する』制御だと言で説明する。
- ・そのコスト関数にL1を足すと入力が0に張り付きやすくなる＝hands-off。
- ・比べるのは3つ：古典のKanayama、標準のL2-MPC、提案のL1-MPC。同じMPCでコスト関数だけ変えてL1の効果を見る。
- ・指標は4つ（精度・エネルギー・スパース性・計算負荷）。

★強調（自分用）：

- ・★ 『同じMPCでコスト関数だけ変える』＝L1の効果を“フェアに”分離する工夫だ、と言添える。
- ・★ 指標は4つを口頭で1つずつ。特に“計算負荷”は後で効いてくる伏線。

スライド14 全体の流れ [目安 0:50]

話す：

- ・この1枚で全体像を見せる。左から右へ進行順。
- ・机上のsim→実機の基盤づくり→古典制御で自立走行→MPC実装→実機で3者比較、という5ステップ。
- ・『実機の基盤が思ったより早く整ったので、計画を前倒しして実機実験まで進めた』と補足。

★強調（自分用）：

- ・★
ここで“全体地図”を頭に入れてもらう。以降のスライドが今どこの話を各スライド冒頭のラベルで示す。

スライド15 机上シミュレーションー 原理の確認 [目安 1:10]

話す：

- ・最初に机上で原理を確認した話。運動モデルを作り、『目標に着く入力のうち操舵を一番使わないもの』を最適化で探した。
- ・右の図がポイント。左は誤差が0に収束＝ちゃんと到達。右が操舵入力で、78%の時間は棒がゼロ＝操舵していない。
- ・つまり“要所だけ動かして、あとは手放し”でも到達できる、という原理確認。
- ・ただし一度きりの計算なので外乱に弱い。だから次はMPC、と繋ぐ。

★強調（自分用）：

- ・★ 右図の『棒が立っていない時間＝操舵していない』を指さして説明すると一発で伝わる。
- ・★ 78%という数字を強調。『これがhands-offの正体です』。
- ・★ オープンループの限界（外乱で誤差が溜まる）を必ず言い、MPCへ自然につなぐ。

スライド16 実機の基盤づくり [目安 1:00]

話す：

- ・地味だが重要な土台づくり。市販ロボットLiteRoverに、自分の制御を載せて走らせる環境を整えた。
- ・LiDARを付けてSLAMで地図も作成（右図）。自己位置と地図を同時に作る技術だと言補足。
- ・PCとロボットの無線通信が不安定だったので、全部ロボット上で動かす方式に切り替えて回避した。
- ・設計の工夫として、制御の計算部分と通信部分を分け、あとでL1に差し替えても使い回せるようにした。

★強調（自分用）：

- ・★ 『ここは“何をやったか”の自立性アピール。自分で環境を作った』点に軽く触れてよい。
- ・★ 通信を“あえて捨てて回避”した判断は、研究を前に進める工学的判断としてプラスに語る。

- ・★ 地図の図は『ちゃんと動く実機がある』証拠として見せる。

スライド17 実機で自立走行（古典制御）〔目安 1:00〕

話す：

- ・比較の基準になる古典制御（Kanayama法）をまず実装して、実機で床を走らせた。
- ・Kanayama法は『経路からのずれを見て操舵を決める』素直なフィードバックだと言で説明。
- ・直線3本とL字を完走。ただしL字で旋回角が 10° 足りなかった。
- ・原因は車輪の滑りと車輪間隔の設定ずれ。定数を補正してほぼ 90° に直した、と話す。

★強調（自分用）：

- ・★ 『実機を動かすと、理論では出ない誤差が出る』ことの最初の具体例。後半のチャタの伏線になる。
- ・★ 回転オドメトリの弱さは、後で“真値は外部計測が必要”の話に繋がる。ここで軽く種をまく。

スライド18 計算が間に合うかの確認〔目安 1:00〕

話す：

- ・MPCは毎周期、最適化を解く。非力なRaspberry Piで0.1秒以内に解けないとリアルタイムに動かない。
- ・だから本格実装の前に“そもそも解けるか”を確認した。これが通らないと研究が成立しない最初の関門。
- ・予測の長さNを変えて求解時間を測った結果、N=30まで余裕で周期内、電圧低下もなし。通過。
- ・提案のL1はL2より重い、それでも間に合う。

★強調（自分用）：

- ・★ 『L1は操舵を減らす、計算は増える』というトレードオフを、ここで数字で初出しする。
- ・★ この“計算負荷”は本研究の貢献の柱の一つ。後の3者比較でもう一度回収する。

スライド19 MPCの実装(1)：L2-MPC〔目安 1:00〕

話す：

- ・ここからMPC本体。まずMPCを一言で：1.5秒先まで予測して経路に一番沿う入力を計算し、先頭だけ実行、を毎周期繰り返す。
- ・その骨格を実装してシミュレーションで検証。5ケース全部で到達した。
- ・結果、経路からのずれRMSEが基準のKanayamaを全ケースで下回った。
- ・“予測して最適化する”分だけ上手、という話。この標準形L2をまず土台にする。

★強調（自分用）：

- ・★ L1の効果を測る“ものさし”を先に作る、という研究の組み立てを強調。
- ・★ 数字（特にL字 11.6→2.2cm）を1つ口に出すと説得力が出る。

スライド20 MPCの実装(2)：L1で疎にする〔目安 1:10〕

話す：

- ・L2とL1の違いを直感で：L2は二乗で罰するので小さな操舵を満遍なく使いなめらか。L1は絶対値で罰するので多くがピタッと0になり疎。
- ・コスト関数を1行変えるだけで切替できる＝同じ条件で公平に比べられる。
- ・重み λ を振って調べた。 λ を上げると操舵は減るが追従は甘くなるトレードオフ。
- ・右の図、左下が良い領域。 $\lambda \approx 0.25$ あたりでL2より少し良い所に入る。

★強調（自分用）：

- ・★ 『二乗 vs 絶対値』のイメージは他分野でも刺さる。手で“満遍なく”と“ピタッと0”を表現するとよい。
 - ・★ ただし正直に：無外乱のシミュレーションだとL2との差は小さい。差が本当に出るのは外乱下や曲線、という見立てを述べておく（次のスライドの伏線）。
-

スライド21 実機で3者比較 (1) : 直線 [目安 1:10]

話す :

- いよいよ実機で3者を比べた。直線2m。1本ごとにメジャーで実測して“真の精度”を取った。
- 結果、提案のL1が一番正確で横ずれ0cm。L2が4cm、古典のKanayamaが14~15cm。
- 計算負荷はKanayama $\approx 0 < L2 < L1$ 。3つとも周期内。
- 大事な注意：ロボット自身の自己位置“だけ”だとL2が良く見える。でもそれは滑りを見られないから。本当の精度は外から測らないと分からない、と気づいた。

★強調（自分用） :

- ★
『提案手法が実機の真値で一番良かった』はポジティブな山。ただし次のL字で話が変わる、と引きを作る。
- ★ “自己位置では分からない・外部計測が要る”は方法論的に効く気づき。研究の丁寧さとしてアピール。
- ★ $n=1$ （1本ずつ）なので『L1 $\times$ L2は今後反復して確かめたい』と正直に添える（過剰主張を避ける）。

スライド22 実機で3者比較 (2) : L字 [目安 1:20]

話す :

- 直線では差が小さかったので、操舵が要るL字でも比べた。
- ここが今日いちばんの見どころ。実機では提案のL1が、曲がり角で操舵を左右に激しく振動させた=チャタリング。
- 数字で：操舵総量がL2の4.5倍、向きの反転が15回。中央の図、赤が上限の間を何度も往復しているのが見える。
- L2はきれいに1回で曲がれた。L1は蛇行して最終的な向きがずれた。

★強調（自分用） :

- ★ ここを一番強く：『スパーズ制御が活きたと思った曲線で、逆にL1だけが破綻した』という“予想と逆”の結果。
- ★ そして『これはシミュレーションでは起きず、実機で初めて見えた』点を強調。これが今回の最大の学び。
- ★ ただしスライドでは断定的に言いすぎない。口頭で温度感を出す（驚き→なぜ？と引っぱって次へ）。
- ★ 図の赤い往復を指で追うと聴衆に伝わる。

スライド23 原因の特定と対策 [目安 1:20]

話す :

- なぜ振動したのかを突き止めた。理想のシミュレーションでは起きないので、実機だけにある“遅れ”を疑った。
- 計算と通信で約0.2秒の遅れがある。これをシミュレーションに足すと、L1だけが実機そっくり振動した。数値もほぼ一致。
- 仕組みは、L1は“ゼロか最大か”の切替的操舵になりやすく、反応が遅れると行き過ぎ→戻し過ぎで振動する。L2はなめらかなので平気。
- 対策として『操舵を急に变えすぎない』項をMPCに足した。シミュレーション上は振動がほぼ消え、しかも疎さは保てた。

★強調（自分用） :

- ★ 『問題を見つけて終わり』ではなく、原因まで突き止めて対策まで示した、という“研究としての完結度”を強調。
- ★ 『シミュレーションと実機の数値が一致』=原因特定が確かだ、という根拠を丁寧に。
- ★ 実機での対策検証は“これから”だが、日程の話はしない（聞かれたら口頭で『次にやる』程度）。
- ★ 他分野向けに『遅れがあると切替制御は振動しやすい』は一般性のある教訓として話すと響く。

スライド24 まとめ [目安 1:00]

話す：

- ・最後にまとめ。やったことは6ステップ（机上sim→基盤→古典で走行→MPC実装→実機3者比較→原因と対策）。
- ・分かったことは4つ。MPCは古典より正確。提案L1の効きは条件しだい（直線で最良・曲線で振動）。その振動は実機の遅れが原因で対策の見込みも得た。そして操舵減と計算増のトレードオフを実機で数値化した。
- ・現時点で安定して良いのはL2。提案のL1は条件しだいで、引き続き実機で検証中、という整理で締める。

★強調（自分用）：

- ・★ ここでは断定や強い言葉を避け、淡々と。『L1が一番』とは言わない。条件しだい・検証中、と誠実に。
- ・★ 一番伝えたい一文は『シミュレーションでは見えず、実機で初めて分かった』。最後にもう一度だけ口頭で言う。
- ・★ 質問が来やすいのは“なぜL2が良いのにL1を研究するのか”。→
省エネや外乱への強さは条件で出るはずで、それを見極めるのが目的、と答える準備。

スライド25 補足の入口【目安 0:15】

話す：

- ・ここからはプログラムを1つずつ詳しく見る（論文解説で触れたコードの“深掘り”）。
- ・時間や質問に応じて、見せる枚数は調整する。

★強調（自分用）：

- ・★ 『制御の中身を自分で実装した』ことが伝わればよい。深入りしすぎない。

スライド26 プログラム全体の構成【目安 1:00】

話す：

- ・全体の流れ。自己位置odom → follower_coreで参照点と誤差 → mpc_coreで最適化して速度 → ロボットへ。
- ・中核は自作のfollower_coreとmpc_core。ほかに検証・解析のスクリプトも自作した。
- ・言語はPython、最適化はcvxpyとOSQP、ROSはROS2。

★強調（自分用）：

- ・★ 『制御の中身も検証ツールも自分で実装した』ことを伝える。
- ・★ “線形MPCはQPとして解ける／将来CasADiで非線形も”は質問が来たら補足。

スライド27 コード解説 follower_core【目安 1:00】

話す：

- ・土台のfollower_core。経路上の目標点に対するずれを、ロボットから見た誤差に直す関数。
- ・Kanayama法は、その誤差に比例ゲインを掛けて速度と角速度を決める素直な制御。予測はしない。
- ・この誤差計算はMPCでもそのまま使い回す共通土台。

★強調（自分用）：

- ・★ 『古典制御は今のずれだけで決める＝予測なし』を一言。次のMPCとの対比になる。
- ・★ コードは雰囲気伝わればOK。1行ずつ読まない。

スライド28 コード解説 スパース最適制御の原理：cvxpy【目安 1:00】

話す：

- ・予備実装したスパース制御の最小実装。cvxpyで『目標に着く入力のうち操舵の総量が最小のもの』を、数式そのままに書ける。
- ・L1ノルムを最小化すると多くの入力が厳密に0になるのが数学的な性質。結果、操舵の78%が0。

★強調（自分用）：

- ・★『 $L1 = \text{絶対値の和}$ を最小化すると0が増える』が、スパース制御の核。ここは少し丁寧に。
- ・★ cvxpyだと数式に近い形で書ける利点も一言。

スライド29 コード解説 MPC本体・L2/L1の切替 : mpc_core [目安 1:10]

話す：

- ・提案手法の心臓部、MPC本体。Nステップ先まで予測して『経路ずれ+操舵ペナルティ』の合計を最小化する。
- ・★の1行だけがL1とL2の違い。norm1ならL1（疎）、quad_formならL2（なめらか）。
- ・同じ骨格でこの1行だけ変えるので、L1の効果をフェアに比べられる。毎周期、係数だけ差し替えて高速に解き直す。

★強調（自分用）：

- ・★『違いはこの1行だけ』が一番の見せ場。指さして言う。
- ・★ “同じ枠組みで公平に比較” という研究設計の妥当性をここで主張。
- ・★ self等はスライドでは省略している、と一言断る。

スライド30 コード解説 チャタ対策：移動抑制 [目安 0:50]

話す：

- ・L字で見つかった振動への対策コード。1つ前の操舵との差を二乗で罰する＝急な操舵変化を抑える項を足しただけ。
- ・既定は0で、足さなければ今までと完全に同じ（後方互換）。simでは振動が反転19回→1回に減った。

★強調（自分用）：

- ・★『問題を見つけて、対策コードまで書いた』完結度を示す。
- ・★ 実機での対策確認はこれから、と正直に（日程は触れない）。

スライド31 コード解説 制御ループ : mpc_follower [目安 1:00]

話す：

- ・実機で毎周期回している制御ループ。0.1秒ごとに、自己位置→参照点→誤差→MPCで速度→ロボットへ、を実行。
- ・★の1行をkanayama_cmdに変えるだけでKanayama版になる。だから3者を同じ枠組みで比較できる。
- ・odomが途切れたり求解に失敗したら止まる安全機構も入れた。

★強調（自分用）：

- ・★『この1行を差し替えるだけ』＝比較の公平性をもう一度強調。
- ・★ 安全機構に触れると“実機をちゃんと扱っている”印象になる。

スライド32 コード解説 検証・解析ツール [目安 0:50]

話す：

- ・実機に載せる前後で使った検証・解析ツール。解けるか測るbench、simで確かめるtest、 λ を振るsweep、遅延でチャタを再現するdelay_probe、実走データを数値化するanalyze_bag。
- ・“いきなり実機”ではなく、simで確かめてから実機、実機の結果を数値で評価、というサイクルを回した。

★強調（自分用）：

- ・★『simと実機を行き来して検証した』丁寧さをアピール。
- ・★ delay_probeが“原因特定”の決め手だった点に触れてもよい。

想定質問と返し

- **なぜ L2 が良いのに L1 (提案) を研究する？** → L1の利点 (省エネ・アクチュエータ保護・外乱への強さ) は条件しだいでは出ない。それを実機で見極めるのが目的。今は“どこで効くか”を切り分けている段階。
- **L1 は計算が重いのでは？** → その通り。だからこそ「操舵は減るが計算は増える」トレードオフを“数値で”示すことが貢献。
- **外乱での評価は？** → これから実施する (傾斜・初期ずれ大・滑りやすい床など)。スパーズ制御の利点はそこで出る見込み。
- **Kanayama 法とは？** → 1990年に提案された古典的な経路追従フィードバック。経路からのずれを見て操舵量を定める素直な方式。比較の基準に使った。
- **実装は何語／どのライブラリ？** → Python。最適化は cvxpy で記述し、実機の求解は OSQP (線形MPCは QP として解ける)。ROS は ROS2(rclpy)。cvxpy を選んだのは目的関数を数式に近い形で書け、L2⇄L1 の切替が1行で済むから。将来は非線形MPCを CasADi+IPOPT で試す予定。
- **コードは全部自作？** → 中核 (誤差計算・MPC・実機ノード) と検証/解析ツールは自作。ロボットのモータ制御とオドメトリは LiteRover 公式サンプルを流用した。
- **チャタの対策は実機で確認した？** → シミュレーションで有効性は確認済み。実機での確認は次のステップ。
- **論文 (永原 2017) と本研究の違いは？** → 論文はオープンループの定式化。本研究はそれを毎周期解き直す MPCに拡張し、補正 δu の L1 を最小化した点と、実機 (遅延あり) で検証して理想との差を示した点の違い。

締めの一言

- 「シミュレーションでは見えず、実機で初めて分かったことが多くありました。引き続き実機で検証していきます。」
- 「ご質問・ご助言をいただければ幸いです。」